

Bashon: A Hybrid Crowd-Machine Workflow for Shell Command Synthesis

Yan Chen, Jaylin Herskovitz, Walter S. Lasecki, Steve Oney

University of Michigan, Ann Arbor, United States

{yanchenm, jayhersk, wlasecki, soney}@umich.edu

Abstract—Despite advances in machine learning, there has been little progress towards creating automated systems that can reliably solve general purpose tasks, such as programming or scripting. In this paper, we propose techniques for increasing the reliability of automated systems for program synthesis tasks via a hybrid workflow that augments the system with input from crowds of human workers. Unlike previous hybrid workflow systems, which have been focused on less complex tasks that crowd workers can do in their entirety (e.g., image labeling), our proposed workflow handles tasks that untrained crowd workers cannot do alone (i.e., scripting). We evaluate our approach by creating BashOn, a system that increases the performance of an automated program that generates Bash shell commands from natural language descriptions by $\sim 30\%$. Our approach can not only help people make program synthesis tools more robust, reliable, and trustworthy for end-users to use, but also help lower the cost of downstream data collection for program synthesis when a preliminary model exists.

Index Terms—program synthesis; crowdsourcing; crowd workflows

I. INTRODUCTION

Bash is a complex but powerful Unix shell and user interface. Users can parameterize and combine Bash commands to quickly perform complex operations that would take much longer in a Graphical User Interface (GUI). However, correctly writing a Bash command requires complex reasoning skills and years of practical experience. Even experienced Bash users often need to rely on support from various resources like Stack Overflow or man pages [1], [2] for recalling code syntax.

Although researchers have created automated systems to help users write Bash scripts and programs from natural language descriptions [3]–[5], they are limited to accomplishing constrained tasks. In our tests with Tellina [4], a state-of-the-art automated tool that translates natural language queries found in the wild into Bash commands, we found a 10% accuracy rate. Part of this is because currently, most automated systems only work reliably in domains where there is sufficient, well-organized training data [6] and the “output” of the system is simple [7]. The only reliable source for support in complex domains like Bash scripting is from human experts, who can be difficult to find.

In this paper, we propose leveraging non-expert crowd workers to boost the accuracy of an existing program synthesis system (Tellina [4]) for generating Bash commands from natural language descriptions. We show that despite not having expertise with Bash scripting, crowd workers can increase the

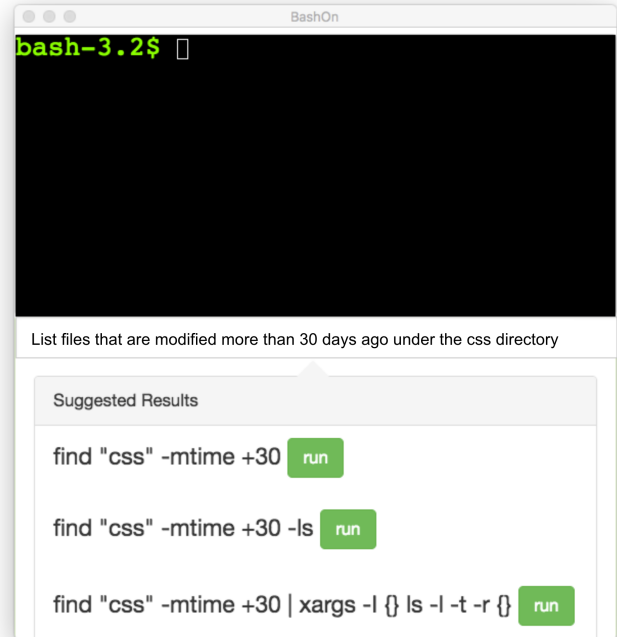


Fig. 1. Bashon allows users to create Bash commands from natural language. It uses a hybrid workflow that integrates crowd workers and an AI for program synthesis. In this example, the user types a command in natural language: “List files that are modified more than 30 days ago under the css directory” and Bashon proposes three Bash commands. The user can press ‘Run’ to execute any of the proposed commands

effectiveness of the fully automated system by nearly 30%. We also introduce Bashon (Fig. 1), a user interface that uses our hybrid workflow to allow users to generate Bash commands from natural language descriptions.

An important feature of Bashon is that it relies on crowd workers who do not have experience with Bash commands. This is in contrast with previous hybrid workflow systems, which rely on crowd workers’ ability to perform a task in its entirety [8]–[12]. Thus, when using non-expert crowd workers, these systems are limited to domains that have relatively low complexity (i.e., little prior knowledge is required). A key insight when designing Bashon is that although non-expert workers are less effective than an automated system at generating Bash commands, they can be effective in targeted portions of its workflow. For example, although a given crowd

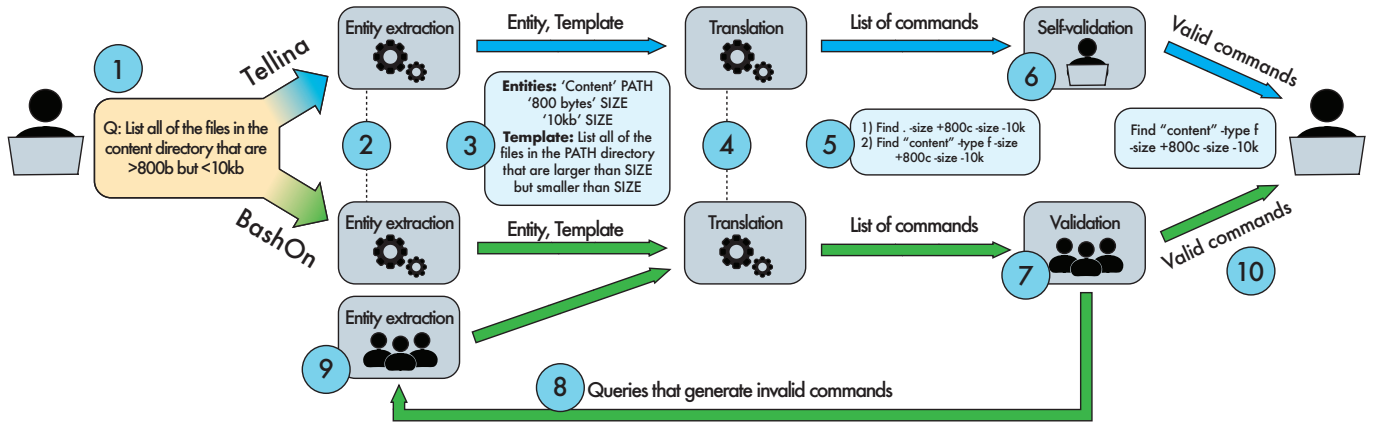


Fig. 2. System diagram: This diagram plots both the original Tellina (blue) and Bashon’s (green) workflow. In Tellina’s workflow, first, a natural language query (1) is sent to an automated entity extraction process (2) which outputs a list of entities, their associated types, and the entity extracted query template (3). Then, the neural network (4) translates the query and the extracted entities into a list of commands (5) that is sent back to the user (6). In Bashon’s workflow, the natural language queries first go through the original Tellina system, with automated entity extraction (1-5). The results from this are then evaluated by crowd workers (7). If they are incorrect (8), they then go to crowd workers for entity extraction (9). Commands resulting from this re-extraction process go back to the end users (10).

worker might not be able to determine what a given Bash command does, they can still help the system by determining whether the *result* of a candidate matches the users’ intentions. They can also support the system by leveraging their contextual understanding to further assist in specifying commands.

We explored multiple ways to restructure the hybrid workflow and then we compared their performance. Our experiments demonstrated that, although the crowd struggled at times with pieces of domain-specific information, overall they were able to make up for places where the automated system was lacking. Beyond Bash commands, our approach provides a broader solution to support complex tasks like programming by directing non-expert effort to evaluation and refinement tasks with minimal modifications to the “black box” components of the intelligent systems.

This paper contributes:

- Methods, challenges, and opportunities in implementing the hybrid workflow in a program synthesis task.
- Evidence that integrating crowd workers into automated system workflows improves performance on complex tasks.
- Bashon, an interactive system that instantiates this idea of a hybrid workflow in the domain of script programming.

II. RELATED WORK AND PROBLEM BACKGROUND

Our work builds on prior work in hybrid intelligence, crowd workflows, and program synthesis.

A. Hybrid Intelligence Workflows

Crowdsourcing can be used to improve machine learning by performing tasks that automated systems cannot do well (e.g., entity extraction and filtering [13], [14]), validating machine-generated results [15], and creating new training data from which the automated systems can learn [11], [12]). Previous studies with these automated systems show the

promise of hybrid workflows, but they rely on crowd workers’ ability to perform a task in its entirety. Systems like Tohme and Zensors [16], [17] leverage both machine learning and crowdsourcing techniques to augment performance in different applications. Tohme uses this combination to detect curb ramps in Google Street View, while Zensors creates sensor feeds that can respond to natural language questions. In addition, studies have shown that hybrid workflows that interleave human labor and machine intelligence perform better than either component alone [18], [19]. Our work explores a hybrid intelligence workflow to integrate non-expert tasks that need to be tightly scoped within a more complex task that requires domain specific knowledge.

B. Program Synthesis and Programming Support

Programming is a difficult task that requires years of training and practice. Even for experienced programmers, memorizing every syntactic rule is nearly impossible, so they have to rely on existing resources. Many online question answering platforms, like Stack Overflow [1], provide asynchronous support allowing software developers to post questions to a large community. But these sites have many limitations, such as long waiting times to receive a response after posting a request, and non-personalized answers [20]. Other work has used human experts to provide support [21], but finding the right expert can be prohibitively difficult and expensive.

There has been extensive research on how to efficiently synthesize programs from natural language descriptions. Gulwani et al. conducted a series of studies to synthesize spreadsheet scripts by developing Domain Specific Languages (DSLs) and synthesis algorithms to learn user-provided examples [22], [23]. But DSLs are not for programmers to read but for machines, and they are domain-dependent. Systems like Codemend [3] and Tellina [4] aim to translate natural language queries to programs or scripts using automated systems trained with

expert-generated data. But the amount of data that is available to train these systems is still small, which limits the model performance. Our hybrid workflow requires automated modules to operate the translation, but there is no need for a human expert to supervise the process. Furthermore, other automated systems often produce complex programs that are different from what programmers usually write and may require non-trivial manual effort to validate and troubleshoot. Bashon instead leverages human intelligence to validate the program execution outcomes, which we show to be an effective way of improving its performance.

C. Tellina: Synthesizing Bash Commands

To explore the effectiveness of using hybrid workflows for generating Bash commands, we built on Tellina [4], a state-of-the-art fully automated system for synthesizing Bash commands. Tellina’s automated workflow consists of three primary steps (illustrated in Figure 2):

- *Entity Extraction*: This first step finds all of the entities and types in the natural language query. This parser restricts input to context-independent queries (e.g., no “this,” “it”), and often requires users to refine queries using special characters (e.g., putting “” around names, strings, and regular expressions).
- *Language Translation*: This step takes the entity extracted query template and translates it into a ranked list of possible Bash command templates using a Recurrent Neural Network (RNN) (sequence-to-sequence) model. Both the list of templates and the entities extracted from the first step are then forwarded to the Argument Filling step.
- *Argument Filling*: The final step fills the argument slots in the candidate commands with the extracted entities to form the complete script. Users then receive a list of full commands. However, this step does not ensure that commands are safe, valid, or executable.

1) *Performance*¹: The reported accuracies of Tellina are $Acc_F^1 = 30.0\%$, and $Acc_F^3 = 36.0\%$ where Acc_F^k is denoted as the percentage of their test examples for which a correct full command is ranked k^{th} or higher in a list of possible translations. According to the authors, 41 out of 50 incorrect sample commands are caused by the mis-recognition of entities in the first step. In addition, the generated commands need to be validated by the users, and these commands can sometimes be unusual and complex, which makes validation difficult. Furthermore, this process could be very tedious and error-prone when using a large-scale file system.

These performance limitations provide an opportunity to introduce a hybrid workflow as a possible solution. Where Tellina’s parser fails to extract the correct entities from a query, human input could potentially be more accurate. Crowd workers could also help to validate the outcomes by voting

on the execution results, as troubleshooting is a cumbersome process when the file system is large. However, the language translation and argument filling steps require domain-specific knowledge of the syntax of Bash commands, which non-expert crowd workers might not be able to gain in a short amount of time. So, for the translation step, we can continue to leverage Tellina’s automated system.

III. THE BASHON WORKFLOW

The Bashon workflow consists of three steps: entity extraction by the crowd, automated language translation, and outcome validation by the crowd. The language translation component is inherited from Tellina. Here, we will explain how we designed and implemented the other two components. Our system architecture (Fig. 2) is similar to Tellina’s; however, we replace the two automated modules with crowd modules. We then hypothesize that these two integrated modules can each enhance the performance of Tellina and lead to an even greater improvement when both are used together.

A. Entity Extraction by the Crowd

Prior work has shown promising results on leveraging crowd workers for named-entity recognition tasks [24], [25]. With regard to scripting tasks, understanding the context and semantics of a query is crucial to extracting the right entities. Thus, our first module utilizes non-expert crowd workers to extract entities from a natural language query. Workers are asked to write down all of the entities in the query and select their associated types. After computing the majority-voted entities and extracting them from the query, the query template is sent to be translated into the ranked command templates.

B. Outcome Validation by the Crowd

Our second module asks crowd workers to assist in validating commands that Tellina has returned. Normally, developers need to validate the commands from Tellina manually, which presents a few issues: Tellina gives no guarantee that the commands are executable, and sometimes the commands can be risky or have unintended side effects if the developers do not carefully review them (e.g., contains “rm”). Thus, users are often frustrated by the syntactic errors present in the generated commands. Tellina also sometimes suggests unusually complex commands (e.g., long pipelines), and their user study participants found them “distracting even if some of these commands were correct” [4].

We instead ask crowd workers to validate and filter candidate commands by determining if the outcomes of these commands match the end-user’s intention. For commands that require validating a large number of results (such as commands that are run on a large-scale file system), we decompose the verification step into small portions and distribute the reduced verification tasks to different crowd workers (Fig. 3. (4)). Each worker is then asked to use the given file system information to either reject or accept the outcome of each command.

¹Tellina’s original report only has their model evaluation results, which is what we reported in this section. Our final evaluation used and compared the “real task” performance which they collected for their user study but without any data cleaning process.

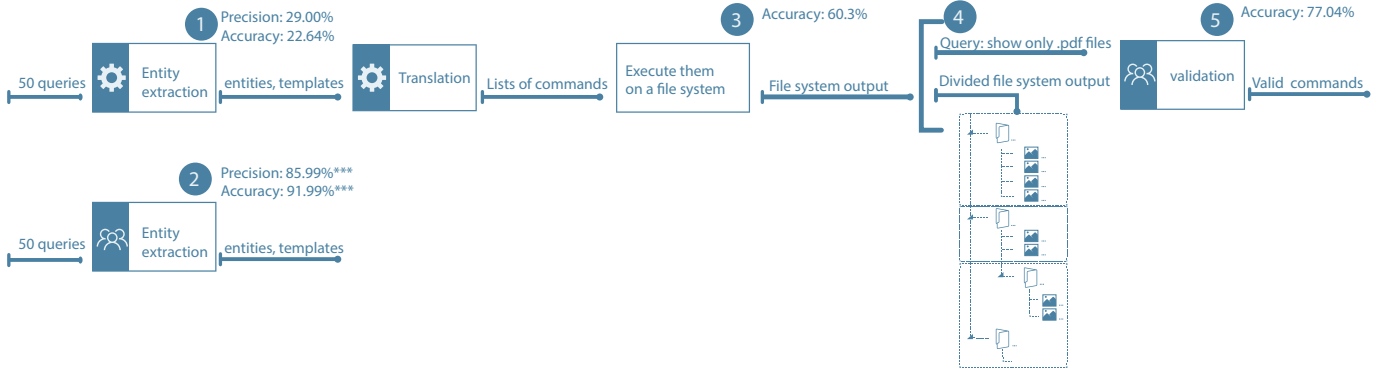


Fig. 3. A pipeline diagram that shows the precision and accuracy between each Tellina component and Bashon's crowd component. (1) Tellina's entity extraction (EE) component performance; (2) Bashon's EE component performance; (3) Tellina's command validation component performance (filters out commands that generate error messages); (4) both the query (e.g., show only pdf files in my folder) and the execution output of the command on a file system, with 20 (or 21) files, were present to the crowd workers; (5) Bashon's validation component performance.

IV. EVALUATION

To evaluate the effectiveness of our approach, we tested three conditions to examine the accuracy of each of the two crowd input modules, individually and together as Bashon, compared with the accuracy of using Tellina alone (Table I).

A. Dataset

We adapted the queries used in Tellina's original experiment—real queries selected from online resources (e.g., Super User)—by having researchers rewrite them to avoid the correct answer from being found online through a keyword search. However, we only used queries that did not aim to modify the file system or any of its contents, as our outcome validation step required us to run the candidate commands on an actual file system in parallel. Ten queries were left after filtering. To increase the validity of our task set, we recruited two Bash users from Upwork (with 2+ years of Bash command experience) to rephrase these 10 queries in different ways (e.g., Fig. 2), as if they were asking the questions themselves. After this, we had 50 queries in total.

B. Condition 1: Tellina (Baseline)

To establish a baseline, we entered all 50 queries into Tellina and computed the performance of its entity extraction (Fig. 3. (1)), as well as the precision of the top 1, 3, and 10 results (Table I). The correct entities were extracted by two authors independently, and as there can be more than one entity in each query, we computed both the precision and accuracy.

Fig. 4. User interface of entity extraction task. Crowd workers can add one entity at a time by typing the entity they recognize in the query to the text box (1), and then select the type from a drop down list (2).

	S1: Tellina (baseline)	S2: Entity Extraction by the Crowd	S3: Validation by the Crowd	S4: BashOn
Accuracy/ Precision (Top 1)	10% (n=50)	28.00%* (n=50)	10% (n=40)	38.29%** (n=28)
Accuracy (Top 3)	17.28% (n=46)	45.71%** (n=46)	21.38% (n=29)	71.43%** (n=21)
Precision (Top 3)	11.11% (0.27,n=46)	32.38%** (0.39,n=46)	13.79% (0.50,n=29)	46.03%** (0.29,n=21)
Accuracy (Top 10)	18.60% (n=44)	51.52%** (n=44)	-	-
Precision (Top 10)	10% (0.22,n=44)	23.03%* (0.27,n=44)	-	-

Table I. Accuracy is the percentage of queries that had a correct answer in the top k commands returned, and precision is the percentage of the top k commands returned that were correct. S3, S4 had fewer than 10 commands returned after filtering. n is the number of queries left at the end. Compared to baseline: * significant at $p < .05$; ** significant at $p < .01$

Note that the accuracy of Tellina [4] reported by Lin et al. was higher than it is in our experiment because our set of queries has looser constraints than the set that they used [4]. The commands are judged based on their execution outcome (e.g., the list of file names), as there could be many correct commands and different representations of the outcome, so they are difficult to validate automatically. The correct execution outcome is then compared with the correct output that was manually generated by the researchers.

C. Condition 2: Crowd-Powered Entity Extraction

We divided the 50 queries into 10 Human Intelligence Tasks (HITs), with each HIT containing five queries. To reduce any potential bias, we randomized the queries to minimize the chance that a worker would receive a rephrased version of a query they have already seen. Although prior studies like [25] have used the crowd to extract entities, it was unclear to us how well the crowd would be able to understand the task, as many of the queries contained domain-specific information. Thus, we iterated on this task by asking the crowd to provide feedback. The issues that were most often raised were the confusing

definitions for each entity and the excess of tasks in one HIT, as we had seven queries per HIT at the beginning. We then clarified the definition of each entity type in the instructions and only released five queries per HIT to increase worker accuracy and decrease mental fatigue (See Fig. 4 for task user interface).

In the final setting, we provided definitions of each entity type, two detailed examples, and graphical instructions on how to use our interface. We recruited 100 crowd workers using a 99%-acceptance-rate filter. We then asked each worker to extract all possible entities for five queries, and we had ten workers parse each query. They spent about 45 seconds (S.D. = 30.11) on average, and we paid \$0.24 for each query. For each task, there were 1.86 (S.D. = 0.61) entities on average. After removing less precise answers (e.g., answers with a non-existent word in the query), we computed the majority votes (6+/10). Figure 3 (2) shows the precision and accuracy, including our finding that the non-expert crowd outperforms the automated process from the original workflow (1). In addition, the accuracy of the top 1, 3, and 10 answers all significantly increased by approximately 18%, 28%, and 33% respectively compared to the baseline condition.

D. Condition 3: Crowd-Powered Outcome Validation

The file system we chose for this study is the same one included in the original Tellina user study (61 files, 4 level deep), as its large size allowed us to analyze the efficacy of our validation strategy. We divided the file system into three portions, with each containing 20 (or 21) files, as we found in our preliminary study that workers could sufficiently handle this amount. To reduce redundant work, we added a filter into our workflow before the execution results were sent to the crowd. We used both heuristics and static analysis [26] to filter out obviously error-prone commands (e.g., those that generated errors) before sending them to crowd workers.

We had nine workers vote on each outcome, and each worker was given both the query, the outcome list, and one third of the user’s original file system (20 or 21 files). Workers were asked to either disapprove (if they found evidence from the given file system that the command outcome was incorrect) or approve (if they could not find such evidence) of each outcome. The outcomes that were not rejected by any crowd workers were considered correct.

We created 180 tasks (18 HITs, 3 assignments each), with each containing 10 commands’ execution results and their original queries. We repeated each of these tasks for each portion of the file system for a total of 540 tasks. There were on average 9.06 candidate commands per query (453/50), and workers spent 23 seconds (S.D = 12.44) on average on each one. We paid workers \$0.13 for each validated execution result for a total of \$1.30 per HIT (an effective rate of $\sim$ \$20/hr).

We then took a majority vote per file portion and aggregated the votes from all three portions. Together with the filter, this validation component was able to correctly validate 77.04% of commands, as shown in Figure 3. (5). The performance of this study is on par with the baseline condition, and we found

that a portion of the queries were filtered out at the end as the crowd identified their results invalid.

E. Condition 4: Bashon

Finally, to evaluate how both crowd components perform jointly, we used the list of queries we generated in our second study (S2) and ran them through Bashon’s full workflow, which is illustrated in Figure 2. Queries were first run through automated entity extraction and translation components, which would save effort if they generated correct results. The results were then validated by the filter and crowd workers. If any of the possible outcomes were incorrect, the query was sent back to the entity extraction step, which was performed by crowd workers this time. The results from this extraction were used again to translate the query with the automated component and then re-validated by workers.

After this re-validation, we were left with two sets of potentially correct commands. We chose to sort this list first by the entity extraction method (Tellina’s automated system, or crowd workers), and then by the number of crowd workers that voted to validate a command (with the max being three). We ranked the commands whose entities were extracted by crowd workers higher, as our second study found human input to have better results.

We used the same number of workers for each part as described in the previous two studies. Table I shows the accuracy and precision of the top 1 and 3 answers after aggregation. Overall, we found that Bashon outperformed all the other three workflows and significantly increased the accuracy by 40% and 30% in the top 1 and 3 answers respectively. We also noticed that not all queries had answers returned by Bashon due to the low translation accuracy, and Bashon’s final answers for some queries consisted of less than 10 candidate commands due to our filter and crowd validation.

V. DISCUSSION

Our experiments demonstrated that crowd workers without domain knowledge can help improve the performance of natural language shell scripting tasks. This includes checking and interpreting command inputs and outputs to make sure a command is behaving as expected, and providing constraints to improve the effectiveness of existing program synthesis techniques by marking the incorrect outputs. Our show the potential of using non-experts to support data collection for complex tasks with a larger and more available crowd than existing methods, and our approach is inexpensive, requiring \$0.63 (\$0.24 + \$0.13 $\times$ 3) and 68s (45s + 23s) per query. In this section, we reflect on our experiments and discuss challenges and opportunities for future work.

A. Challenges and Solutions in Translation

Complex tasks like scripting can be difficult for both humans and Artificial Intelligence (AI) systems to master, in part because of the complexity and the large variation of the input data. In our study, we found that crowd workers can extract entities more accurately than the automated parser across a

variety of paraphrases of the same query. We believe this is due to the fact that the automated parser does not leverage the context when extracting, whereas humans are generally capable of understanding the context and semantic meaning of the query. This situation often occurs when an entity must be understood within the context of the query. For example, one set of queries in our study asked the program to find some files in the "content" directory, which the automated parser misunderstood as a request to show the content of the files. Another example would be if the query "Copy file A to path B" was phrased instead as "Under my path B, make a copy of file A"; the automated system would make the opposite prediction of which entity type is which. We propose that the crowd could assist in finding the correct order of the entities in a query by using information from the appropriate man page. From the second example above, we could first pull out the description of "copy" (i.e., cp: Copy SOURCE to DEST) and ask the crowd to mark the entity order that matches the user's intent.

An additional difficulty is that the initial queries themselves can contain errors or lack key pieces of information. Neither Bashon nor Tellina offer any features that allow users to clarify or get notifications regarding incomplete queries (i.e., missing an entity). For example, the query "Sort my PDF files" misses a few pieces of information (e.g., "By what?", "Under which directory?"), which prevents any method from reliably generating the right answer. Our proposed solution to this issue is to provide similar example queries and man page templates to crowd workers. Prior work [27] has shown that providing such gold standard examples to crowd workers can increase the quality of their responses. Since the AI system has a training data set, we could find similar queries from this set, present them to crowd workers, and ask them to vote on the completion of the query. If the crowd has access to man page information, they may be able to identify missing entities and ask for clarification from the user. This would make the system more friendly to novice Bash users, as they currently may not realize when something is missing from their requests.

B. Analysis and Challenges in Entity Extraction

Tellina's original study found that their system had some difficulty extracting certain semantic types, such as a date formatted like "the 2nd of August of 2017," or a directory named "content," as they were using heuristics to try to cover all of these cases. However, we found that the crowd could identify these entities accurately. On the other hand, some phrases require domain knowledge to understand, and so the crowd also had some difficulties performing the task in a different way. For example, we found that workers identified the phrase "long-iso format" as a date, a file name, or a pattern (when it is actually not an entity), which led to suboptimal outcome. Similar errors like identifying the word "regular" from the request "List all the regular files..." as a file name, were fairly frequent. We believe this is also due to a lack of knowledge of Linux queries, despite our efforts in defining different entity types. We propose two possible ways of addressing this issue:

- 1) having crowd workers request clarification from the user, or
- 2) providing crowd workers with more examples from the training data.

However, we also noticed the expertise level of crowd workers varies, and future work could place more emphasis on responses from workers with more expertise.

Additionally, some extracted entity lists might look correct in the voting mechanism we have currently, but they could contain small errors. Mistakes like typos, excluding the units from a size type entity, or including the word "directory" in a file type entity are easy to interpret as correct answers. These types of misinterpretations could easily become a majority voted entity.

C. Analysis and Challenges in Validation

As some queries contained metadata constraints (e.g., size), we appended the command `-exec ls -lh \;` after each command before executing, as it provides the time and size information of each file, allowing workers to evaluate the outcome. This approach can be generalized in future work to cover a broader set of queries: we could allow the crowd workers to self-identify extra information they need during the validation test, and the system could adapt the commands accordingly.

One challenge present in our validation study is that the generated commands may modify the files, increasing the difficulty of validation. Although we only use queries that do not have the intent to modify the file system, incorrect commands may still arise, and malicious intent or bad judgment from the crowd in the entity extraction step could lead the system to generate problematic commands. One way to deal with this issue is to make copies of the file system for each command and execute the commands on the copy of the system. This preserves the original file system, but the computational cost is very high. Another solution is to identify risky commands like `rm` in the results before running them. This proactively prevents modification from happening, but it requires a pre-defined list of risky commands, which is also inefficient. We instead set the permissions of each file to read-only, preventing non-administrator users from modifying it. Future work in this domain can look into finding a way to safely execute commands in an efficient way, possibly with an interface that can generate the outcome of multiple commands executing on the same file system.

Another challenge is that certain commands may appear incorrect but actually generate the desired outcome. In this case, our approach might be able to even outperform an experienced Bash command user's judgment. However, it is also possible that commands could be incorrect while returning the correct results by chance in the examples observed (i.e., they do not generalize, or have unintended side effects). Future work could explore how crowd-returned commands can be validated by the user easily from a scripting perspective.

Additionally, some commands might generate excessive information, like a long list of file names, which would be cumbersome for any human worker to validate. We considered limiting the length of the outcome, but for some queries long

output could be desirable. Consequently, to design tasks without overloading workers with information while still covering a variety of query types, we propose dividing the outcome into pieces in the future and distributing them to multiple workers, similar to our division of the file system.

The voting strategy that we used would fail to validate the results of queries that request a portion of a file system relative to other parts. For example, take the query “Find the second largest file in my root directory.” If the file system is divided into three portions where the first portion contains the largest file, second portion contains the second largest file, and the third portion contains the third largest file, none of the crowd workers could validate the result as the requested information is relative to what the rest of the system looks like.

Therefore, we propose a mechanism where we ask the crowd to report evidence they found based on their portion of the file systems that lead them to decide not to disapprove of the given outcome. This data would then be aggregated and voted on by crowd workers a final time. Using the same example above, the expected evidence from the crowd would be the number of files with sizes greater or equal to the return file, so there would be two files shown as evidence in the aggregation step. If the executed command returned the second one of these files, then it would be correct. Not only would this be a good method for allowing a larger range of commands to be validated, but it could also serve as a double check of the commands’ validity to ensure no mistakes have been made for other types of commands where we would not normally have to aggregate evidence.

D. Challenges in the Workflow

Even if the all entities are correctly extracted, the correct command may still not be generated. We argue that this is an issue with the original machine learning model, as even when gold standard input data is used, sometimes incorrect output is still generated. It is also possible that a modified version of our workflow could be more accurate than what we are currently using. As described previously, our workflow has a single loop in which queries are sent back to the crowd, after which commands from crowd workers and the automated system are aggregated. Changing our workflow so that it loops multiple times or only loops when the number of incorrect commands generated is above a certain threshold are possible alternatives. Additionally, the generated commands could be ranked by different factors as well. Future work could explore the effect that variations like this have on accuracy, precision, and cost.

E. Using Non-expert Crowd for Data Collection

Although non-expert crowd workers alone are not capable of answering these queries to produce the training data set needed to improve an AI model, we showed that with the existing automated system they could generate valid data (i.e., query and command pairs) with certain accuracy. In the future, we plan to investigate whether having the non-expert crowd help with the program synthesis process would lighten the workload

for experienced script users and still generate useful training data.

VI. CONCLUSION

In this paper, we implemented and evaluated a hybrid intelligence workflow that augments an existing program synthesis system to translate natural language query to Bash commands by leveraging non-expert crowd users. Our experimental results showed that despite not having expertise in the domain of Bash scripting, crowd workers can significantly increase the accuracy when integrated into targeted portions of this workflow. Our approach sheds light on ways that other program synthesis tools might leverage non-expert crowds to power more reliable systems.

VII. ACKNOWLEDGEMENTS

We thank Victoria Lin, first author of the Tellina system, for the feedback on this work, and sharing data with us. We thank Sang Won Lee and Aubrey Ahmed for their editing assistance, our anonymous reviewers for their helpful suggestions on this work, and our study participants for their time.

REFERENCES

- [1] S. Overflow, “Stack overflow, <https://stackoverflow.com/>,” 2015, accessed: April, 2016. [Online]. Available: <https://stackoverflow.com/>
- [2] M. Page, “Man page, en.wikipedia.org/wiki/manpage,” 2015. [Online]. Available: en.wikipedia.org/wiki/Manpage
- [3] X. Rong, S. Yan, S. Oney, M. Dontcheva, and E. Adar, “Codemend: Assisting interactive programming with bimodal embedding,” in *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*. ACM, 2016, pp. 247–258.
- [4] X. V. Lin, C. Wang, D. Pang, K. Vu, and M. D. Ernst, “Program synthesis from natural language using recurrent neural networks,” Tech. Rep., 2017.
- [5] A. Desai, S. Gulwani, V. Hingorani, N. Jain, A. Karkare, M. Marron, R. Saitesh, and S. Roy, “Program synthesis using natural language,” in *Software Engineering (ICSE), 2016 IEEE/ACM 38th International Conference on*. IEEE, 2016, pp. 345–356.
- [6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *Computer Vision and Pattern Recognition, 2009. CVPR 2009. IEEE Conference on*. IEEE, 2009, pp. 248–255.
- [7] A. Ng, “What artificial intelligence can and can’t do right now,” 2016, <https://hbr.org/2016/11/what-artificial-intelligence-can-and-cant-do-right-now>.
- [8] J. P. Bigham, C. Jayant, H. Ji, G. Little, A. Miller, R. C. Miller, R. Miller, A. Tatarowicz, B. White, S. White *et al.*, “Vizwiz: nearly real-time answers to visual questions,” in *Proceedings of the 23rd annual ACM symposium on User interface software and technology*. ACM, 2010, pp. 333–342.
- [9] S. W. Lee and *et al.*, “Sketchexpress: Remixing animations for more effective crowd-powered prototyping of interactive interfaces,” in *UIST*. ACM, 2017.
- [10] W. S. Lasecki, J. Kim, N. Rafter, O. Sen, J. P. Bigham, and M. S. Bernstein, “Apparition: Crowdsourced user interfaces that come to life as you sketch them,” in *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*. ACM, 2015, pp. 1925–1934.
- [11] S. Iyer, I. Konstantas, A. Cheung, J. Krishnamurthy, and L. Zettlemoyer, “Learning a neural semantic parser from user feedback,” *arXiv preprint arXiv:1704.08760*, 2017.
- [12] R. A. Cochran, L. D’Antoni, B. Livshits, D. Molnar, and M. Veanes, “Program boosting: Program synthesis via crowd-sourcing,” in *ACM SIGPLAN Notices*, vol. 50, no. 1. ACM, 2015, pp. 677–688.
- [13] S. Swaminathan, R. Fok, F. Chen, T.-H. K. Huang, I. Lin, R. Jadhani, W. S. Lasecki, and J. P. Bigham, “Wearmail: On-the-go access to information in your email with a privacy-preserving human computation workflow,” 2017.

- [14] D. Merritt, J. Jones, M. S. Ackerman, and W. S. Lasecki, "Kurator: Using the crowd to help families with personal curation tasks." 2017.
- [15] J. Cheng and M. S. Bernstein, "Flock: Hybrid crowd-machine learning classifiers," in *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing*. ACM, 2015, pp. 600–611.
- [16] K. Hara, J. Sun, R. Moore, D. Jacobs, and J. Froehlich, "Tohme: detecting curb ramps in google street view using crowdsourcing, computer vision, and machine learning," in *Proceedings of the 27th annual ACM symposium on User interface software and technology*. ACM, 2014, pp. 189–204.
- [17] G. Laput, W. S. Lasecki, J. Wiese, R. Xiao, J. P. Bigham, and C. Harrison, "Zensors: Adaptive, rapidly deployable, human-intelligent sensor feeds," in *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*. ACM, 2015, pp. 1935–1944.
- [18] J. W. Vaughan, "Making better use of the crowd," 2016.
- [19] J. Attenberg, P. G. Ipeirotis, and F. J. Provost, "Beat the machine: Challenging workers to find the unknown unknowns." 2011.
- [20] L. Mamykina, B. Manoim, M. Mittal, G. Hripcsak, and B. Hartmann, "Design lessons from the fastest q&a site in the west," in *Proceedings of the SIGCHI conference on Human factors in computing systems*. ACM, 2011, pp. 2857–2866.
- [21] Y. Chen, S. Oney, and W. S. Lasecki, "Towards providing on-demand expert support for software developers," in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. ACM, 2016, pp. 3192–3203.
- [22] S. Gulwani, W. R. Harris, and R. Singh, "Spreadsheet data manipulation using examples," *Communications of the ACM*, vol. 55, no. 8, pp. 97–105, 2012.
- [23] S. Gulwani, "Automating string processing in spreadsheets using input-output examples," in *ACM SIGPLAN Notices*, vol. 46, no. 1. ACM, 2011, pp. 317–330.
- [24] A. Ritter, S. Clark, O. Etzioni *et al.*, "Named entity recognition in tweets: an experimental study," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2011, pp. 1524–1534.
- [25] T.-H. K. Huang, W. S. Lasecki, and J. P. Bigham, "Guardian: A crowd-powered spoken dialog system for web apis," in *Third AAAI conference on human computation and crowdsourcing*, 2015.
- [26] "Shellcheck, <http://www.shellcheck.net/>," 2017, accessed: Sep, 2017. [Online]. Available: <http://www.shellcheck.net/>
- [27] S. Dow, A. Kulkarni, S. Klemmer, and B. Hartmann, "Shepherding the crowd yields better work," in *Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work*. ACM, 2012, pp. 1013–1022.